KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

Hotel Room Reservation and Guest Management System

Submitted by

KISHOR KUMAR K M

25BIT026

Under the Guidance of

DHILEEBAN A

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026 Semester: **I**

DECLARATION

I hereby declare that the project entitled "**Hotel Room Reservation and Guest Management System"** submitted in partial fulfillment of the requirements for the course **JAVA PROGRAMMING** is a **record of my original work** carried out under the guidance of , Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

KISHOR KUMAR K M

(Your Name)

CERTIFICATE

This is to certify that the project entitled **Hotel Room Reservation and Guest Management System"** is a **bona fide work** carried out by **[KISHOR KUMAR K M] [I-BSc IT-.'A']**, **[]**, **KG College of Arts & Science**, in partial fulfillment of the requirements for the course **[JAVA PROGRAMMING]**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: COIMBATORE

Date: 09.04.2026

Submitted for the Viva-Voce Examination held on **KG COLLAGE OF ARTS & SCIENCE**

Guide:
(Technical Trainer, KGCAS)

Head of the Department
(Technical Head, KGCAS)

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	
6	**Input and Output Screens**	
7	**Discussion and Conclusion**	
7.1	Discussion	
7.2	Conclusion	
8	**References**	

ABSTRACT

The Hotel Room Reservation and Guest Management System is a Java-based application designed to automate the process of loan management. In many financial institutions and small lending environments, loan verification and EMI tracking are handled manually, which often leads to calculation errors, delays in processing, and inefficient record management. This project aims to simplify loan handling by providing an automated system that verifies eligibility and tracks EMI payments effectively.

The system allows users to enter personal and financial details, check loan eligibility based on predefined conditions, calculate EMI, and monitor payment status. It ensures that users clearly understand their repayment schedule and remaining balance, helping in better financial planning and avoiding missed payments. The application is developed using Core Java programming concepts such as classes, objects, methods, loops, conditional statements, and basic data structures.

The main objective of the system is to improve loan processing efficiency, reduce human errors, and provide accurate and timely financial information. The system displays clear loan details including loan amount, EMI, tenure, and outstanding balance, enabling better decision-making.

This project demonstrates the practical implementation of object-oriented programming concepts in Java and provides a strong foundation for developing more advanced banking and financial management systems in the future.

1. INTRODUCTION

The Loan Eligibility Verification and EMI Payment Tracking System is a Java-based console application.

It helps users determine whether they are eligible for a loan based on income.

The system also calculates the EMI using standard financial formulas.

Users can track their monthly EMI payments and remaining balance.

Manual loan calculations are often time-consuming and error-prone.

This system simplifies the entire loan management process.

It ensures accuracy, speed, and efficiency in financial calculations.

The project is useful for learning real-world Java applications.

1.1 Objective

1. The main objective is to automate hotel room booking and guest management.
2. It reduces manual work and improves efficiency.
3. Helps in maintaining accurate guest records.
4. Provides quick room availability checking.
5. Simplifies reservation and cancellation processes.
6. Reduces errors in billing and data entry.
7. Enhances customer satisfaction through faster service.
8. Maintains proper check-in and check-out records.
9. Improves data storage and retrieval.
10. Supports better decision-making for hotel management.

1.2 Overview

1. The system is a Java-based application.
2. It manages room reservations and guest details.
3. Users can book, update, and cancel reservations.
4. Stores guest information securely.
5. Displays available and occupied rooms.
6. Calculates billing automatically.
7. Maintains records in a structured format.
8. Reduces paperwork in hotels.
9. Easy to use interface for staff.
10. Suitable for small and medium hotels.

1.3 Features

1. Room booking functionality.
2. Guest registration system.
3. Check-in and check-out management.
4. Room availability tracking.
5. Billing and invoice generation.
6. Data storage and retrieval.
7. Search guest details easily.
8. Reservation cancellation option.
9. Simple user interface.
10. Error handling and validation.

-

2. PROGRAM ANALYSIS

Program analysis involves understanding the problem and defining how the system will solve it effectively.

---

2. PROGRAM ANALYSIS

2.1 Problem Definition

1. Manual booking leads to errors.
2. Difficulty in tracking room availability.
3. Loss of guest records.
4. Time-consuming reservation process.
5. Billing mistakes are common.
6. Poor data management system.
7. Lack of proper documentation.
8. Difficulty in updating records.
9. Inefficient customer service.
10. Need for automation in hotel operations.

2.2 Project Overview

1. The system automates hotel operations.
2. It manages reservations and guest details.
3. Provides user-friendly interface.
4. Stores data in structured format.
5. Allows easy modification of records.
6. Reduces human errors.
7. Supports quick data access.
8. Improves efficiency of hotel staff.

9. Ensures accurate billing.
10. Enhances customer experience.

2.3 Scopes and Limitations

1. Scope includes reservation and guest management.
2. Can be used in small hotels.
3. Improves operational efficiency.
4. Reduces manual workload.
5. Provides basic reporting features.
6. Limitation: console-based interface.
7. No online booking feature.
8. Limited database storage.
9. No advanced analytics.
10. Requires manual data input.

-

3. SYSTEM DESIGN

System design defines how the system components are organized and interact.

3.1 Modules

1. Login Module (optional).
2. Room Management Module.
3. Guest Management Module.
4. Reservation Module.
5. Billing Module.
6. Check-in/Check-out Module.
7. Data Storage Module.
8. Search Module.
9. Update/Delete Module.
10. Report Generation Module.

4. METHODOLOGY

Methodology describes the process followed to develop the project. It explains the steps taken from understanding the problem to designing, implementing, testing, and documenting the system.

For the **Hotel Room Reservation and Guest Management System**a structured development approach was followed to ensure that the system meets the requirements and functions efficiently.

The development process includes the following stages:

• Problem Analysis
• System Design
• Program Development
• Testing and Validation
• Result and Documentation

This section explains the steps followed to develop the system.

4.1 Problem Analysis

1. **Study current hotel system.**
2. **Identify problems in manual process.**
3. **Analyze requirements.**
4. **Determine system needs.**
5. **Understand user expectations.**
6. **Identify inputs and outputs.**
7. **Analyze data flow.**
8. **Define system boundaries.**
9. **Study feasibility.**
10. **Prepare solution approach.**

4.2 System Design

1. **Design system architecture.**
2. **Identify modules and components.**
3. **Create data flow diagrams.**
4. **Design database structure.**
5. **Plan user interface.**
6. **Define input/output formats.**
7. **Ensure modular design.**
8. **Apply object-oriented principles.**
9. **Plan error handling.**
10. **Finalize system blueprint.**

4.3 Program Development

1. **Use Java programming language.**
2. **Implement classes and objects.**
3. **Develop modules step by step.**
4. **Write clean and structured code.**
5. **Use methods for functionality.**
6. **Handle exceptions properly.**
7. **Integrate all modules.**
8. **Test each function.**
9. **Debug errors.**
10. **Complete final implementation.**

4.4 Testing and Validation

1. **Perform unit testing.**
2. **Test each module individually.**
3. **Conduct integration testing.**
4. **Validate inputs and outputs.**
5. **Check for errors and bugs.**
6. **Ensure correct functionality.**
7. **Test different scenarios.**
8. **Verify system performance.**
9. **Fix identified issues.**
10. **Confirm system reliability.**

4.5 Result and Documentation

1. **System works successfully.**
2. **Generates accurate results.**
3. **Stores data efficiently.**
4. **Reduces manual errors.**
5. **Improves hotel management.**
6. **Documentation prepared clearly.**
7. **Includes user manual.**
8. **Provides system details.**
9. **Explains working process.**
10. **Useful for future reference.**

4.6 Algorithm

1. **Start the program.**
2. **Display menu options.**
3. **Accept user choice.**
4. **Perform selected operation.**
5. **Input guest/room details.**
6. **Check room availability.**
7. **Store or update data.**
8. **Generate bill if required.**
9. **Display results.**
10. **End program.**

4.7 Flowchart

1. **Start process.**
2. **Display menu.**
3. **User selects option.**
4. **Decision making step.**
5. **Input data.**
6. **Process data.**
7. **Store or retrieve data.**
8. **Display output.**
9. **Repeat if needed.**
10. **End process.**

5. SOURCE CODE

There are three modules are there they are

module 1.

```
public class Guest {

    int id;

    String name;

    String phone;


    public Guest(int id, String name, String phone) {

        this.id = id;

        this.name = name;

        this.phone = phone;

    }

}
```

module 2.

```
import javax.swing.*;

import java.awt.*;

import java.awt.event.*;


public class HotelGUI extends JFrame implements ActionListener {


    JButton addGuestBtn, bookRoomBtn, viewRoomsBtn, checkoutBtn;

    HotelService service = new HotelService();


    public HotelGUI() {


        setTitle("Hotel Management System");
```

```
        setSize(400, 350);
        setLayout(new GridLayout(4, 1, 10, 10));
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        addGuestBtn = new JButton("Add Guest");
        bookRoomBtn = new JButton("Book Room");
        viewRoomsBtn = new JButton("View Rooms");
        checkoutBtn = new JButton("Checkout");

        add(addGuestBtn);
        add(bookRoomBtn);
        add(viewRoomsBtn);
        add(checkoutBtn);

        addGuestBtn.addActionListener(this);
        bookRoomBtn.addActionListener(this);
        viewRoomsBtn.addActionListener(this);
        checkoutBtn.addActionListener(this);

        setVisible(true);
    }

    public void actionPerformed(java.awt.event.ActionEvent e) {

        if (e.getSource() == addGuestBtn) {
            try {
                int id =
Integer.parseInt(JOptionPane.showInputDialog("Enter Guest ID:"));
```

```
                String name = JOptionPane.showInputDialog("Enter
Name:");
                String phone = JOptionPane.showInputDialog("Enter
Phone:");

                JOptionPane.showMessageDialog(this,
                        service.addGuest(id, name, phone));

            } catch (Exception ex) {
                JOptionPane.showMessageDialog(this, "Invalid Input!");
            }

        } else if (e.getSource() == bookRoomBtn) {

            try {
                int id =
Integer.parseInt(JOptionPane.showInputDialog("Enter Guest ID:"));
                JOptionPane.showMessageDialog(this,
service.bookRoom(id));
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(this, "Invalid Input!");
            }

        } else if (e.getSource() == viewRoomsBtn) {

            JOptionPane.showMessageDialog(this, service.viewRooms());

        } else if (e.getSource() == checkoutBtn) {

            try {
```

```
                int roomNo =
Integer.parseInt(JOptionPane.showInputDialog("Enter Room Number:"));
                JOptionPane.showMessageDialog(this,
service.checkout(roomNo));
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(this, "Invalid Input!");
            }
        }
    }

    public static void main(String[] args) {
        new HotelGUI();
    }
}
```

module 3.

```
import java.util.*;

public class HotelService {

    ArrayList<Guest> guests = new ArrayList<>();
    ArrayList<Room> rooms = new ArrayList<>();

    public HotelService() {
        for (int i = 1; i <= 5; i++) {
            rooms.add(new Room(i));
        }
    }

    public String addGuest(int id, String name, String phone) {
```

```
        guests.add(new Guest(id, name, phone));
        return "Guest Added Successfully!";
    }

    public String bookRoom(int id) {
        Guest selected = null;

        for (Guest g : guests) {
            if (g.id == id) {
                selected = g;
                break;
            }
        }

        if (selected == null) return "Guest Not Found!";

        for (Room r : rooms) {
            if (!r.isBooked) {
                r.isBooked = true;
                r.guest = selected;
                return "Room " + r.roomNumber + " Booked!";
            }
        }

        return "No Rooms Available!";
    }

    public String viewRooms() {
        StringBuilder status = new StringBuilder();
```

```
        for (Room r : rooms) {
            if (r.isBooked) {
                status.append("Room ").append(r.roomNumber)
                        .append(" - Booked by
").append(r.guest.name).append("\n");
            } else {
                status.append("Room ").append(r.roomNumber)
                        .append(" - Available\n");
            }
        }

        return status.toString();
    }

    public String checkout(int roomNo) {
        for (Room r : rooms) {
            if (r.roomNumber == roomNo && r.isBooked) {
                r.isBooked = false;
                r.guest = null;
                return "Checkout Successful!";
            }
        }
        return "Invalid Room!";
    }
}
```

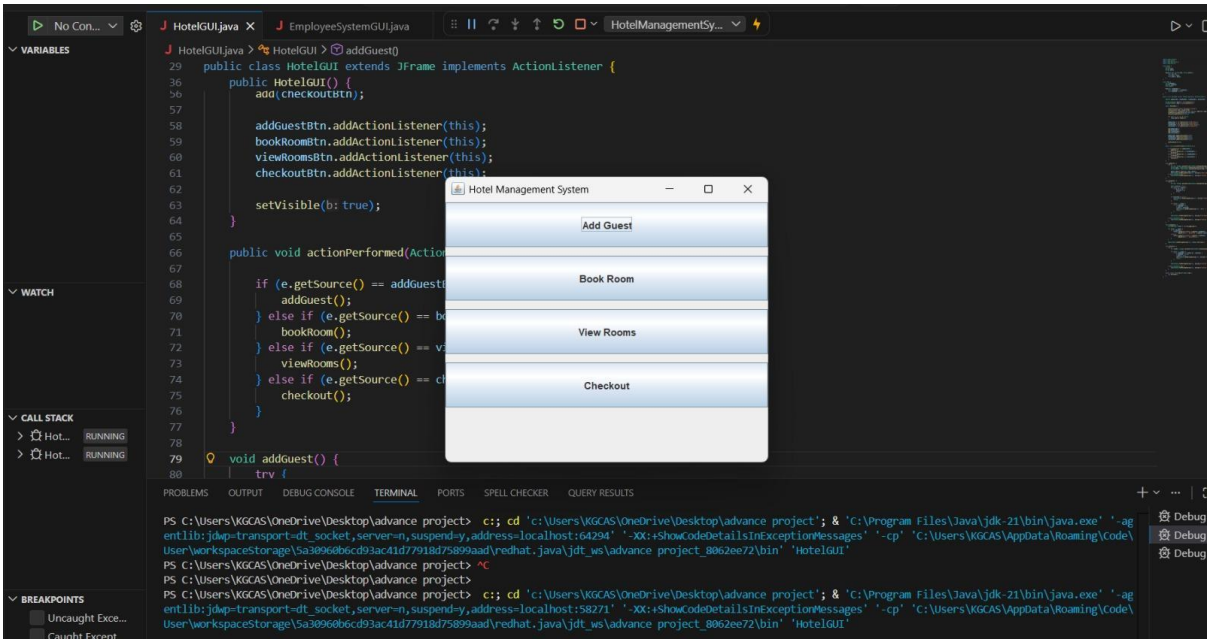
Module 4.

```
public class Room {
```

```
    int roomNumber;
    boolean isBooked;
    Guest guest;


    public Room(int roomNumber) {
        this.roomNumber = roomNumber;
        this.isBooked = false;
    }
}
```

6. INPUT AND OUTPUT SCREENS



7. DISCUSSION & CONCLUSION

7.1 Discussion

1. System reduces manual effort.
2. Improves accuracy in booking.
3. Easy to use interface.
4. Efficient data management.
5. Faster operations.
6. Reliable performance.
7. Useful for hotel staff.
8. Minimizes errors.
9. Enhances customer service.
10. Practical implementation of Java concepts.

7.2 Conclusion

1. The project is successfully implemented.
2. It automates hotel management tasks.
3. Reduces errors and saves time.

4. Improves efficiency of operations.
5. Easy to use and maintain.
6. Provides accurate results.
7. Suitable for small hotels.
8. Can be enhanced further.
9. Demonstrates Java programming skills.
10. A useful real-time application.

8. REFERENCES

The following resources were used during the development of this project:

Books

• Herbert Schildt – *Java: The Complete Reference*

Websites

• https://www.oracle.com/java
• https://www.geeksforgeeks.org
• https://www.w3schools.com
• https://www.javatpoint.com